

From patient registration to verified laboratory report

A practical map of the hybrid laboratory record system: who acts, what happens next, and when a result becomes available.

Scope: Phase One academic prototype. This guide describes the repository implementation and its documented operating modes. Use synthetic or formally de-identified records.

1	Register patient Registration / Cashier	Create or locate the patient record; link a patient login when portal access is needed.
2	Consult and request Doctor	Record the consultation and create the laboratory request with selected tests or panels.
3	Classify payment Registration / Cashier	Record Paid, Free, Waived, or Subsidized before specimen accession.
4	Handle specimen Laboratory Staff	Assign accession/specimen identifiers, collect the sample, and record receipt or rejection.
5	Test and check quality Laboratory Staff	Enter results, record quality control (QC), and resolve repeats or referrals before submission.
6	Submit and approve Staff → Supervisor	Submit a result version. A different authorized user reviews and approves, or rejects with a reason.
7	Store and register Automated services	Encrypt the approved snapshot, store the package, and register its proof. Save both receipts.
8	Release report Laboratory Supervisor	Release the registered version. The ledger release receipt enables the report and verification URL.
9	Review, access, verify Doctor / Patient / Public	The doctor records review; the patient sees their released results; QR verification shows redacted proof.

Read from top to bottom. Exception paths return work for correction or hold publication until a failed step is resolved.

Each role has a defined handoff

Role	Workflow responsibility	Handoff / boundary
System Administrator	Manage accounts, roles, and operational oversight.	No automatic clinical superuser access.
Registration / Cashier	Manage patient identity, account linking, visits, and payment classification.	Patient and payment data ready for the clinical team.
Doctor	Record consultations, request tests, and review released reports for authorized patients.	Review is a separate event after release.
Laboratory Staff	Accession and handle specimens; enter results; manage QC, repeats, referrals, and submission.	Every required item must be ready for verification.
Laboratory Supervisor	Approve or reject submitted results; release registered versions.	Cannot approve a version they submitted.
Patient	Sign in to view their own released records.	Account must be linked to the patient record.
Public Verifier	Open the opaque QR verification URL.	Anonymous proof lookup; no clinical record access.

Normal order status sequence

REQUESTED → PAYMENT_CLASSIFIED → ACCESSIONED
→ COLLECTED → IN_TESTING → FOR_VERIFICATION
→ APPROVED → STORAGE_PENDING → STORED
→ LEDGER_PENDING → LEDGER_REGISTERED → RELEASED

Before submission

All required test items need a passing QC attempt or an incorporated referral result. Required repeats must be resolved.

Before release

The result version must be VERIFIED and its publication status REGISTERED. An initial order must be LEDGER_REGISTERED.

Status distinction: order status, result-version status, and publication status are separate. The UI can show an approved order as “Completed” while storage or ledger work is still pending. Only release makes it available as a released result.

Access starts with authentication and role checks. Seed accounts must complete their required password change before protected workflows.

How approval becomes a public proof

The browser reaches four services through the gateway. Authentication owns identity and sessions; Records & Sync coordinates the laboratory workflow. Each service owns its data.

1. Records & Sync: approve the immutable version

Commit approval, audit/history, and a STORE_APPROVED_RESULT outbox event. Build the canonical result snapshot.



2. Encrypted Storage: protect and store the snapshot

Create an AES-256-GCM package with a fresh data key and nonce. Store the encrypted envelope in private IPFS, or the labeled filesystem adapter.



3. Records & Sync: save the storage receipt

Retain the ciphertext hash and protected object reference for the exact record/version, then enqueue REGISTER_LEDGER.



4. Verification: register proof and return a receipt

Register opaque record/version and integrity metadata through Fabric Gateway, or the labeled file ledger. Records advances to LEDGER_REGISTERED.



5. Supervisor + services: release the registered version

The supervisor requests release. Verification records the ledger release and issues an opaque token and URL; Records saves the receipt and marks the version RELEASED.



6. Separate access paths: private report and public verification

Authorized doctors and the linked patient access the released report. /verify/<opaque-token> checks token validity and the registered proof.

What the public verifier sees

Verification status, opaque record identifier, version, issuing organization, approval/release timestamps, and ledger label. Patient identity, test names and values, notes, CIDs, and keys are excluded. This is a proof/status check; it does not independently validate clinical accuracy or an uploaded report file.

Native development

File-backed data, encrypted filesystem objects, and a simulated file ledger.

Docker / Fabric

Base Docker uses MySQL and private Kubo. Fabric needs its separate bootstrap; check readiness for fabric and simulated: false.

When the normal flow pauses

Rejected specimen

Record the rejection reason. The order can enter RECOLLECTION_REQUIRED, then return to accession for a new specimen. Do not advance the rejected sample into testing.

QC failure, repeat testing, or referral

Keep the result in laboratory work until required QC passes or an external referral result is incorporated. Resolve the required repeat before submitting.

Supervisor rejection

Keep the rejected version and reason, and create a linked draft. For the initial workflow the order returns to IN_TESTING; the next version needs verification again.

Correction after approval or release

Create a linked new version; repeat QC/submission, approval, storage, registration, and release. Releasing a correction marks the prior released version SUPERSEDED in Records. Do not assume that this automatically revokes an older public token.

Storage, ledger, or release failure

Keep the failed step visible in the outbox. Retries reuse the event idempotency key. The current handler uses increasing delays, capped at one hour, and changes to FAILED_REVIEW after ten failed attempts.

Connection loss or synchronization conflict

Supported drafts and queued commands use IndexedDB. Reconnect and synchronize; review duplicate or revision-conflict receipts. Approval and release require current online server state. Offline synchronization remains a prototype capability.

Invalid, expired, or revoked QR token

Show the verification outcome without exposing patient information. A failed lookup must not be presented as a verified report.

Repository sources

Prepared from documentation and source inspection; no live workflow or infrastructure test was run for this document. The existing E2E scenario provides a synthetic example of the complete journey.

- README.md; docs/ARCHITECTURE.md; docs/OFFLINE_SYNC.md
- services/records-service/src/domain/workflow.js; statuses.js
- services/records-service/src/services/recordsService.js
- services/verification-service/src/services/verification-service.js
- apps/web/src/offline/indexedDb.js
- tests/e2e/specs/full-laboratory-flow.spec.js